
Acceso Remoto Seguro con WireGuard sobre *the-store*

Despliegue, pruebas y alcance acotado de la solución

Trabajo Práctico Especial — Redes de Información

WireGuard (Client-To-Site / Site-To-Site) + Kubernetes NetworkPolicies

Plataforma: Kind (cluster de una sola PC)

Aplicación: the-store (5 microservicios)

Documento: How-To unificado + problemática

Resumen

Esta guía documenta, de punta a punta, una solución de **acceso remoto seguro** al cluster de microservicios *the-store*: cómo desplegarla, cómo probarla y **qué problemas resuelve y cuáles no**. La solución combina **WireGuard** (túneles cifrados Client-To-Site y Site-To-Site) con **Kubernetes NetworkPolicies** (segmentación de red).

El documento está dividido en dos partes. La **Parte I** la problemática. La **Parte II** es el how-to operativo: despliegue paso a paso y demostraciones reproducibles.

Índice

Resumen	1
 Parte I — Problemática	 3
1. Contexto real del deploy	3
2. Problema	3
3. POC y casos de uso	4
 Parte II — How-To: despliegue y pruebas	 5
4. Qué se demuestra	5
5. Prerequisitos	5
6. Arranque rápido (TL;DR)	5
7. Paso a paso	5
7.1. Generar claves	5
7.2. Desplegar el gateway WireGuard	6
7.3. Cliente admin (Client-To-Site)	6
7.4. Red corporativa (Site-To-Site)	6
7.5. Segmentación (NetworkPolicies)	6
7.6. Verificación end-to-end	6
7.7. Rotación de claves (ante compromiso de un peer)	6
8. Cómo funciona por dentro	7
9. Teardown	7

Parte I — Problemática

1 Contexto real del deploy

The Store corre sobre un cluster **Kind** (una sola máquina) con 5 microservicios: UI, Catalog, Cart, Orders y Checkout. El estado verificado sobre el repositorio (`dist/kubernetes.yaml`, `local.sh`) es:

- **Todos los Services son ClusterIP**: no hay NodePort ni LoadBalancer en la aplicación.
- **El único punto de entrada es el Ingress NGINX** publicado en `localhost:80` (sólo se mapean 80/443 del nodo al host).
- **No existen NetworkPolicies de fábrica**: cualquier pod alcanza a cualquier pod.
- **Persistencia in-memory** en los 5 servicios: no hay Redis/Postgres/RabbitMQ desplegados sobre la red.

Consecuencia central: fuera del Ingress en `localhost:80`, el cluster **no es alcanzable desde fuera del nodo de ninguna forma segura**. No hay canal para operarlo remotamente ni para que una red externa consuma sus servicios.

2 Problema

Acotamos el trabajo a **dos problemas de acceso remoto y uno de segmentación** que los habilita de forma segura.

Problema A — No hay acceso administrativo remoto seguro

Un administrador o desarrollador **fuera de la red del nodo** no puede operar el cluster (`kubectl` contra el API server, ni alcanzar servicios internos) sin exponer el API server o los servicios en texto plano. → **Resuelto con WireGuard Client-To-Site**.

Problema B — Una red externa no puede consumir los servicios

Una **red corporativa completa** necesita consumir los servicios del cluster **sin instalar un cliente VPN en cada equipo** y sin abrir el cluster a la red. → **Resuelto con WireGuard Site-To-Site** (túnel permanente entre el gateway corporativo y el del cluster; los equipos finales sólo necesitan una ruta estática).

Problema C — Sin segmentación, el acceso remoto es acceso total

Una vez que el túnel deja entrar tráfico a la red del cluster, **cualquier origen alcanzaría cualquier pod** (movimiento lateral). → **Resuelto con NetworkPolicies** (`default-deny` + reglas mínimas por servicio). Es la pieza de defensa en profundidad que hace que el túnel habilite *sólo* lo necesario.

Qué provee cada tecnología (sin sobre-prometer)

Tecnología	Qué aporta exactamente
WireGuard	Canal de red cifrado (ChaCha20-Poly1305) y autenticado por clave pública entre peers, para los tramos <i>cliente↔gateway</i> y <i>red corporativa↔gateway</i> . AllowedIPs define qué subredes cruzan el túnel (mínimo privilegio de ruteo).
NetworkPolicies	Segmentación a nivel de red del cluster : default-deny y reglas explícitas de qué servicio habla con qué servicio. Bloquea movimiento lateral.

3 POC y casos de uso

Incluye (demostrable, todo en una PC con Kind): gateway VPN del lado del cluster; Client-To-Site (admin → `kubectl` + servicios); Site-To-Site (red corporativa → servicios sin cliente VPN en el equipo final); NetworkPolicies (default-deny + reglas mínimas); validación de tráfico cifrado y rotación de claves.

- CU-1 Admin opera el cluster remotamente (Client-To-Site):** levanta su túnel, hace `kubectl` contra el API server y alcanza servicios internos; sólo llega a las subredes de sus `AllowedIPs`.
- CU-2 Empleado consume servicios sin cliente VPN (Site-To-Site):** desde la red corporativa, el tráfico hacia un servicio cruza el túnel de forma transparente. El equipo final **no tiene WireGuard**.
- CU-3 Segmentación efectiva (NetworkPolicies):** un origen que entró por el túnel **no** puede hablar con un servicio no permitido (p. ej. alcanzar Orders salteándose a UI).
- CU-4 Rotación de claves ante compromiso:** se rota el par de un peer; las claves anteriores quedan invalidadas y el túnel se restablece.

Parte II — How-To: despliegue y pruebas

4 Qué se demuestra

Escenario	Qué prueba
Client-To-Site	Un admin remoto levanta un túnel y opera el cluster con <code>kubectl</code> (API server <code>10.96.0.1</code>) y accede a servicios internos. Todo cifrado.
Site-To-Site	Una red corporativa llega a los servicios sin WireGuard en el equipo final (sólo ruta estática).
NetworkPolicies	<code>default-deny</code> + reglas mínimas: el túnel sólo alcanza lo permitido; el movimiento lateral queda bloqueado.
Cifrado	En el cable sólo se ve UDP cifrado (ChaCha20-Poly1305); el HTTP viaja en claro únicamente dentro del túnel.
Rotación de claves	Se rota un par en caliente; la clave anterior queda invalidada.

5 Prerequisitos

- **Docker**, **Kind** y **kubectl** instalados (ver `the-store-main/README.md`).
- Módulo `wireguard` del kernel (modo primario). Si no está disponible, la imagen cae a **userspace** (`wireguard-go`) automáticamente. El módulo se carga en el kernel **del host** (los pods comparten ese kernel; no tienen kernel propio). Para forzar la carga: `sudo modprobe wireguard`.

6 Arranque rápido (TL;DR)

```
# 1. App de microservicios (cluster Kind + the-store)
cd the-store-main && ./local.sh create-cluster --skip-tests && cd ..

# 2. Toda la solución de acceso remoto, en orden, con verificación
vpn/scripts/up.sh
```

`up.sh` corre, en orden: `00-gen-keys` → `10-deploy-gateway` → `20-up-admin-client` → `25-up-external-pc` → `30-up-site-to-site` → `40-apply-netpol` → `90-verify`. Al final debe imprimir **RESULTADO: 10 PASS, 0 FAIL** (con todos los escenarios levantados).

7 Paso a paso

7.1 Generar claves

```
vpn/scripts/00-gen-keys.sh
```

Crea 4 pares en `vpn/keys/` (gitignored): `gw-c2s`, `gw-s2s`, `admin`, `corp`. Cada privada nunca sale de su lado; sólo las públicas se intercambian.

7.2 Desplegar el gateway WireGuard

```
vpn/scripts/10-deploy-gateway.sh
kubectl get pod -n vpn-system
kubectl exec -n vpn-system deploy/wg-gateway -c wg-gateway -- wg show
```

Crea el namespace `vpn-system`, el Secret con `wg0.conf/wg1.conf` y el Deployment + Service **NodePort UDP** (31820 Site-To-Site, 31821 Client-To-Site). El pod levanta `wg0` y `wg1`, habilita `ip_forward` (initContainer) y NATe a los CIDRs del cluster.

7.3 Cliente admin (Client-To-Site)

```
vpn/scripts/20-up-admin-client.sh
docker exec admin wg show
docker exec admin kubectl --kubeconfig /root/admin.kubeconfig get pods -n the-store
```

El kubectl sale al API server **10.96.0.1 por el túnel** (la ruta a 10.96.0.0/16 sólo existe sobre `wg0`).

7.4 Red corporativa (Site-To-Site)

```
vpn/scripts/30-up-site-to-site.sh
docker exec corp-pc wg show          # vacío: NO tiene WireGuard
CAT=$(kubectl get svc catalog -n the-store -o jsonpath='{.spec.clusterIP}')
docker exec corp-pc curl -s -o /dev/null -w '%{http_code}\n' http://$CAT/health # 200
```

7.5 Segmentación (NetworkPolicies)

```
vpn/scripts/40-apply-netpol.sh
kubectl get networkpolicy -n the-store
```

7.6 Verificación end-to-end

```
vpn/scripts/90-verify.sh          # imprime PASS/FAIL por chequeo
```

7.7 Rotación de claves (ante compromiso de un peer)

```
vpn/scripts/rotate-keys.sh admin  # o: rotate-keys.sh corp
```

Rota el par de claves del peer indicado **sin reiniciar el gateway**:

1. Genera un par nuevo para el peer (`wg genkey / wg pubkey`).
2. Actualiza el peer en el gateway **en caliente** con `wg set`: quita la clave pública vieja y agrega la nueva (la anterior queda **invalidada de inmediato**).
3. Persiste el cambio en el Secret del gateway (sobrevive a un reinicio del pod).
4. Reconfigura el cliente (`admin` o `corp-gateway`) con la clave nueva y reconecta el túnel.

Al final el script verifica que la clave vieja **ya no figura** en el gateway y que la nueva está activa (CU-4). Para comprobarlo a mano:

```
kubectrl exec -n vpn-system deploy/wg-gateway -c wg-gateway -- wg show wg0
docker exec admin wg show # handshake reciente con la clave nueva
```

8 Cómo funciona por dentro

- **Gateway** = **pod** en **vpn-system**, modo **kernel** (sólo **NET_ADMIN**); un **initContainer** privilegiado habilita **ip_forward** (en un pod **/proc/sys/net** es **read-only**). Fallback **userspace** (**wireguard-go**) si no hay módulo.
- **Exposición** = **NodePort UDP**: los clientes (containers en la red **kind**) alcanzan **<nodeIP>:31820/31821**.
- **AllowedIPs** = mínimo privilegio de ruteo y ACL de origen: definen qué subredes cruzan el túnel por cada peer.
- **NAT por destino** en el gateway: todo lo que se reenvía a **10.96.0.0/16** / **10.244.0.0/16** sale con la IP del pod, así el cluster sabe responder (vale para admin, túnel y la LAN corporativa).
- **NetworkPolicies** las aplica el CNI de Kind (**kindnet**): **default-deny ingress** + una regla por servicio.

Todos los valores de red están en **vpn/scripts/lib.sh**.

Mapa de rangos de red

CIDR	Rol
10.200.0.0/24	Subred del túnel Client-To-Site (IPs virtuales de WireGuard; gateway .1, admin .2).
10.100.0.0/30	Subred del túnel Site-To-Site (gateway corp ↔ gateway cluster).
10.96.0.0/16	Service CIDR del cluster (ClusterIPs; API server 10.96.0.1).
10.244.0.0/16	Pod CIDR del cluster (IPs reales de los pods).
172.21.0.0/16	Red Docker kind (nodo y contenedores admin/corp/external).
172.30.0.0/24	LAN corporativa simulada (corp-pc).

9 Teardown

```
vpn/scripts/99-teardown.sh # quita VPN/netpols, deja la app intacta
vpn/scripts/99-teardown.sh --all # ademas borra las claves
```